vLLM Load Testing Guide

1. Repository Setup

Clone the repository, switch to the load testing branch, and navigate to the testing directory:

Clone the repository
git clone https://github.com/ArcaAI/vox-platform-study.git
Navigate into the project folder
cd vox-platform-study
Checkout the target branch
git checkout vllm_load_testing
Move to the load testing directory
cd vllm_load_testing

Note: The `vllm.py` execution script is located inside the `vllm_load_testing` directory.

2. Prerequisites & Environment Setup

Python Environment:

Requires Python 3.10+,Check your Python version:

python3 --version

Option A: Activate Conda Environment

conda activate vox

Option B: Create and Activate Virtual Environment

python3 -m venv .venv

source .venv/bin/activate

Install dependencies:

pip install -r requirements.txt

3. Remote GPU Server Access & Passwordless SSH

1. Remote GPU Server Access & Passwordless SSH
2. The benchmark executes tasks against the remote GPU server at:**192.168.112.2**
3. Generate an SSH key pair: **ssh-keygen -t ed25519 -C "vllm-benchmark-client"**
4. Copy the public key to the remote GPU server:**ssh-copy-id {username}@192.168.112.2**
5. Verify passwordless SSH connection:**ssh -o BatchMode=yes {username}@192.168.112.2 "nvidia-smi"**

4. How to Run `vllm.py`

Notice: Ensure all commands below are executed directly from the `vllm_load_testing` directory.

Run full test:
python vllm.py --config benchmark_config.yaml

Run Only Concurrency 32
python vllm.py --config benchmark_config.yaml --concurrency 32

Run Custom Concurrency Subset with Cooldown
python vllm.py --config benchmark_config.yaml --concurrencies 4,8,16 --cooldown 10

Test Burst Rate with 250 ms Chunks
python vllm.py --config benchmark_config.yaml --pace 0.0 --chunk-ms 250

Run Benchmark Using a Specific Audio File
python vllm.py --config benchmark_config.yaml --audio testfiles/gastro.wav

5. Manual Logging & Diagnostics Runbook

This provides copy-pasteable terminal commands to inspect, monitor, and save Docker container logs and NVIDIA GPU telemetry from the remote GPU host (192.168.112.2).
Note: Replace {username}@192.168.112.2 with your GPU host if using a different server or IP address.

5.1 NVIDIA GPU Monitoring (nvidia-smi):

A. View Current GPU Status:
Inspect GPU utilization, VRAM usage, temperature, and active process PIDs.
ssh {username}@192.168.112.2 "nvidia-smi"

B. Save Point-in-Time Snapshot to File:
Captures both the formatted process table and CSV machine-readable metrics into logs/nvidia_smi.txt.
ssh {username}@192.168.112.2 "nvidia-smi && echo '' && nvidia-smi --query-gpu=timestamp,name,utilization.gpu,utilization.memory,memory.used,memory.total,temperature.gpu,power.draw --format=csv" > logs/nvidia_smi.txt

C. Live Real-Time Dashboard:
Watches GPU activity live in the terminal with a 1-second refresh rate.
ssh -t {username}@192.168.112.2 "watch -n 1 nvidia-smi"

D. Record Continuous Time-Series GPU Telemetry to CSV:
Samples GPU metrics every 50 ms directly into logs/gpu.csv. Press Ctrl+C to stop recording.
ssh {username}@192.168.112.2 "nvidia-smi --query-gpu=timestamp,utilization.gpu,utilization.memory,memory.used,memory.total,power.draw --format=csv -lms 50" > logs/gpu.csv

5.2 Docker Container Logs (vllm-whisper)

A. Follow Live Logs in Real Time:
Tail and follow incoming inference requests as audio chunks arrive.
ssh -t {username}@192.168.112.2 "docker logs -f --tail 100 vllm-whisper"

B. View Recent Logs:
View the last 200 lines without following the log stream.
ssh {username}@192.168.112.2 "docker logs --tail 200 vllm-whisper"

C. Filter Logs by Time Window:
View logs from the last 5 minutes.
ssh {username}@192.168.112.2 "docker logs --since 5m vllm-whisper"

View logs since a specific ISO UTC timestamp.
ssh {username}@192.168.112.2 "docker logs --since 2026-10-26T10:00:00Z vllm-whisper"

D. Save vLLM Docker Logs to Local File:
Save recent runtime logs, for example, the last 500 lines.
ssh {username}@192.168.112.2 "docker logs --tail 500 vllm-whisper" > logs/vllm_audio.log
Save complete container history.
ssh {username}@192.168.112.2 "docker logs vllm-whisper 2>&1 | gzip -c" | gunzip > logs/vllm_audio.log

5.3 Pipeline Engine Logs (hope-stt)

A. Follow Live Engine Logs:
Use this to troubleshoot VAD evaluation, RNNoise suppression, or Redis pub/sub latency on the STT engine.
ssh -t {username}@192.168.112.2 "docker logs -f --tail 100 hope-stt"

B. Save Recent STT Logs to File:
ssh {username}@192.168.112.2 "docker logs --tail 500 hope-stt" > logs/hope_stt.log

5.4 Quick Command Cheat-Sheet

Instant GPU Snapshot:
ssh {username}@192.168.112.2 "nvidia-smi"

Save GPU Snapshot
ssh {username}@192.168.112.2 "nvidia-smi" > logs/nvidia_smi.txt

Record Continuous GPU CSV
ssh {username}@192.168.112.2 "nvidia-smi --query-gpu=timestamp,utilization.gpu,utilization.memory,memory.used,memory.total,power.draw --format=csv -lms 50" > logs/gpu.csv

Stream Live vLLM Logs
ssh -t {username}@192.168.112.2 "docker logs -f --tail 100 vllm-whisper"

Save Recent vLLM Logs
ssh {username}@192.168.112.2 "docker logs --tail 500 vllm-whisper" > logs/vllm_audio.log

Save Complete vLLM Log
ssh {username}@192.168.112.2 "docker logs vllm-whisper 2>&1 | gzip -c" | gunzip > logs/vllm_audio.log

Stream Live STT Engine Logs
ssh -t {username}@192.168.112.2 "docker logs -f --tail 100 hope-stt"

Save Recent STT Engine Logs

ssh {username}@192.168.112.2 "docker logs --tail 500 hope-stt" > logs/hope_stt.log